ASE CHAPTER 7

T2 Descriptive & Short-Answer Pack

Question 7.1 · Software Reliability Definition & Operational Existence (Exercise 7.1)

What is software reliability? Does it exist?

Software reliability is defined as the probability of failure-free software operation for a specified period of time in a specified environment. It is formally expressed as:

$$R(t) = P(\text{No failures occur in the interval } [0, t] \mid \text{specified operational environment})$$

Does it exist? Yes, software reliability genuinely exists as an operational, measurable, and probabilistic property of software execution. Although software is a logical entity that does not suffer from physical wear-and-tear, corrosion, or material fatigue like hardware, software reliability exists because: - **Input sequence uncertainty:** Operational execution selects a dynamic sequence of inputs from an extremely large input space. - **Latent design faults:** Software contains latent specification, design, and coding faults introduced during development. - **Failure manifestation:** When the operational trajectory encounters an input state that activates an uncorrected fault, an observable failure occurs. Software reliability quantifies the probability that the system executes without encountering such fault-triggering input states over a designated operating interval.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة امتحانية: البرمجيات لا تبلى فيزيائياً مثل العتاد، لكن موثوقيتها حقيقية وتُقاس باحتمالية تنفيذ المدخلات من دون تفعيل عيوب برمجية كامنة خلال زمن تشغيل محدد ← التعريف الرسمي المعتمد يربط الموثوقية بالاحتمالية، والزمن، وبيئة التشغيل المحددة.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1 & §7.1.1, pp. 308–310; companion slides 6–8.

Question 7.2 · Software Failure vs Software Fault (Exercise 7.4)

What is software failure? How is it related with a fault?

A software failure is an observable departure of the external system behavior or output from user requirements or specified system behavior during execution. It is a dynamic runtime event that occurs only when the software is actively executed.

A software fault (commonly called a defect or bug) is an internal structural flaw, incorrect step, erroneous process, or incorrect data definition in a software program. It is a static entity that resides within the source code, design specifications, or configuration data.

Relationship between Fault and Failure: - **Causal relationship: A fault is the latent internal cause; a failure is the dynamic external manifestation.** - **Activation requirement:** A fault remains dormant until an input state from the execution environment triggers the faulty code path and propagates incorrect

internal states to the observable system boundary. - **Non-equivalence:** Not every execution of faulty code produces an immediate failure (e.g., if erroneous data is masked or overwritten before reaching the output). Conversely, a single design fault may cause multiple failures across different test runs, or multiple interacting faults may cause a single system crash.

ملاحظة أكاديمية هامة (Academic Note)

الفخ الامتحاني: العيب (Fault) كيان استاتيكي ساكن في الكود ← والفشل (Failure) حدث ديناميكي مرصود أثناء التشغيل ← لا يشترط أن يؤدي كل عيب إلى فشل فوري، لكن كل فشل ينتج عن عيب كامن تم تفعيله بمدخلات معينة.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.2, pp. 310–312; companion slides 8–10.

Question 7.3 · Characterizing Failure Occurrences with Respect to Time (Exercise 7.5)

Discuss the various ways of characterizing failure occurrences with respect to time.

In software reliability modeling, failure occurrences over time are characterized through four distinct mathematical and observational representations:

1. **Time of Failure:**
2. **Recording the exact elapsed operating times $t_1, t_2, t_3, \ldots, t_i$ at which successive failures occur**, measured from the beginning of execution testing ($t = 0$).
3. **Time Interval Between Failures:**
4. **The elapsed operating time T_i between two successive failure occurrences**, defined as:

$$T_i = t_i - t_{i-1}$$

5. This represents failure inter-arrival times and is modeled as a random variable.
6. **Cumulative Failures in a Given Time Interval:**
7. **The total number of failures experienced from time zero up to a specified execution time t**, denoted as $\mu(t)$ (the mean value function) or counting discrete events $M(t)$.
8. **Failure Intensity:**
9. **The instantaneous rate of failure occurrences per unit of execution time**, defined as the derivative of the expected cumulative failure function:

$$\lambda(t) = \frac{d\mu(t)}{dt}$$

10. Typically expressed in units of failures per CPU hour or failures per calendar month.

ملاحظة أكاديمية هامة (Academic Note)

تنبيه: هناك 4 طرق رئيسية لتمثيل حدوث الفشل عبر الزمن: زمن حدوث الفشل الفردي (t_i)، الفترة الفاصلة بين فشلين (T_i)، العدد التراكمي للفشل ($\mu(t)$)، وكثافة الفشل الفورية ($\lambda(t)$) ← السؤال يطلب هذه الأربعة حصراً.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.3, pp. 312–314; companion slides 11–15.

Question 7.4 · Core Reliability Terminology (Exercise 7.6)

Describe the following terms: 1. Operational Profile 2. Input Space 3. MTBF (Mean Time Between Failures) 4. MTTF (Mean Time To Failure) 5. Failure Intensity

1. **Operational Profile:**
2. **A quantitative specification of the complete set of operational tasks or transactions executed by a software system, paired with their respective probabilities of occurrence in actual customer use.**
3. It guides realistic, usage-based reliability testing by ensuring the most frequently executed user scenarios receive proportional test coverage.
4. **Input Space:**
5. **The complete mathematical set of all possible input vectors, data states, and environmental conditions that a software system can receive and process during execution.**
6. Only a tiny fraction of the input space activates latent faults (termed the *failure domain*).
7. **MTBF (Mean Time Between Failures):**
8. **The expected operating time between consecutive failures for a repairable system.**
9. It incorporates both operational operating time and the time required to diagnose, repair, and restart the software:

$$\mathrm{MTBF} = \mathrm{MTTF} + \mathrm{MTTR}$$

10. **MTTF (Mean Time To Failure):**
11. **The expected average operating time of a system from the start of operation until the occurrence of the next failure.**
12. For a constant failure rate λ, it is given by $\mathrm{MTTF} = \frac{1}{\lambda}$.
13. **Failure Intensity (λ):**
14. **The number of software failures occurring per unit of execution time**, representing the instantaneous failure frequency of the system (expressed as failures per CPU hour).

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة: العلاقة الحاكمة بين مؤشرات الزمن: $\mathrm{MTBF} = \mathrm{MTTF} + \mathrm{MTTR}$ ← للأنظمة البرمجية القابلة للإصلاح، الفرق هو زمن الاستعادة والإصلاح (MTTR).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.4–§7.1.7, pp. 314–318; companion slides 16–23.

Question 7.5 · Uses of Reliability Studies in Software Engineering (Exercise 7.7)

What are the uses of reliability studies? How can one use software reliability measures to monitor the operational performance of software?

Reliability studies serve multiple core engineering and management functions across the software lifecycle:

- **Release Readiness Certification:** Providing quantitative, objective criteria to decide when testing can terminate and software is ready for commercial deployment (e.g., reaching a target failure intensity $\lambda \leq \lambda_F$).
- **Testing Progress & Trajectory Tracking:** Evaluating whether reliability is actively growing as faults are repaired, or if testing has plateaued due to inadequate test case diversity or regression defects.
- **Resource and Schedule Estimation:** Projecting the additional execution time, calendar days, and debugging effort required to meet customer quality commitments.
- **Maintenance Planning:** Forecasting field failure rates to estimate customer support staffing, warranty liabilities, and patching schedules.

Monitoring Operational Performance:

- **Operational failure logging:** Measuring the mean time between failures (MTBF) and failure intensity during live production across different customer operational profiles.
- **Threshold alerting:** Comparing observed operational failure rates against service level agreements (SLAs) and baseline reliability models to detect degradation caused by environment changes, hardware shifts, or faulty software updates.

ملاحظة أكاديمية هامة (Academic Note)

الفائدة الهندسية: دراسات الموثوقية تحول قرار إطلاق البرنامج (Release Decision) من مجرد تخمين زمني أو نفاد ميزانية إلى قرار كمي مبني على الوصول لكثافة الفشل المستهدفة ($\lambda \leq \lambda_F$).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.8, pp. 320–322; companion slides 21–23.

Question 7.6 · Software Quality Definition & Quality Attributes (Exercise 7.8)

What is software quality? Discuss software quality attributes.

Software quality is defined as the totality of features and characteristics of a software product or service that bear on its ability to satisfy stated or implied customer needs (ISO/IEEE standard). Conformance to explicit functional requirements, documented development standards, and implicit characteristics expected of professional software.

Core Software Quality Attributes:

1. **Functionality: The degree to which the software provides functions that meet stated user requirements and specifications.**

2. **Reliability: The capability of the software to maintain its level of performance and failure-free operation under stated conditions for a specified period.**
3. **Usability: The effort and ease required for a user to learn, operate, prepare inputs for, and interpret outputs from the software.**
4. **Efficiency: The relationship between the software's performance level and the amount of computational resources (CPU time, memory, bandwidth) utilized.**
5. **Maintainability: The ease with which the software can be modified, corrected, improved, or adapted to environmental changes.**
6. **Portability: The ability of software to be transferred from one hardware/software platform or operating environment to another.**

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة: الجودة مفهوم مظلي شامل (Umbrella Concept) يضم الموثوقية كإحدى أهم سماته ← الموثوقية تجيب عن: "هل يعمل البرنامج دون فشل؟" بينما الجودة تجيب عن: "هل يلبي البرنامج كافة الاحتياجات بكفاءة وسهولة وقابلية صيانة؟".

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2, pp. 321–323; companion slides 24–29.

Question 7.7 · Software Quality Standards: Essence and Benefits (Exercise 7.9)

What do you mean by software quality standards? Illustrate their essence as well as benefits.

ISO 9001 specifies requirements for a quality system and is the standard in the ISO 9000 series most applicable to software development. The Capability Maturity Model (CMM) is a staged model for judging software-process maturity and guiding process improvement; a CMM maturity level is distinct from ISO 9001 certification.

Their practical value is to make quality-system activities documented and reviewable. ISO 9001 addresses areas such as management responsibility, document control, inspection and testing, corrective action, and quality records. CMM organizes software-process practices into maturity levels and supports continuing process improvement.

These documented activities can support consistent execution, traceable records, and process improvement.

ملاحظة أكاديمية هامة (Academic Note)

الفرق الأساسي: ISO 9001 معيار لنظام الجودة ويمكن أن يتطلب السوق شهادة له؛ أما CMM فيقيّم نضج عمليات البرمجيات ويوجه تحسينها، ومستوى النضج مو شهادة ISO.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.1 & §7.4, pp. 323–325, 347–350; companion slides 27–32.

Question 7.8 · McCall Software Quality Model Structure & Product Factors (Exercise 7.10)

Describe the McCall software quality model. How many product quality factors are defined and why?

The McCall Software Quality Model (developed by Jim McCall in 1977 for the US Air Force) organizes software quality into a hierarchical framework connecting high-level management factors, intermediate technical criteria, and concrete metrics.

Structure and Factor Count: McCall defines exactly **11 product quality factors grouped under 3 operational perspectives (lifecycle phases):**

Product Operation (5 factors — operational characteristics during live execution):

- **Correctness:** The extent to which a program satisfies its specifications and user mission objectives.
- **Reliability:** The extent to which a program performs its intended function with required precision over time.
- **Efficiency:** The amount of computing resources and code required to execute a function.
- **Integrity:** The extent to which unauthorized access to software or data can be controlled.
- **Usability:** The effort required to learn, operate, prepare input, and interpret output.

Product Revision (3 factors — modifiability and maintenance characteristics):

- **Maintainability:** The effort required to locate and fix an error in an operational program.
- **Flexibility:** The effort required to modify an operational program to incorporate new requirements.
- **Testability:** The effort required to test a program to ensure that it performs its intended function.

Product Transition (3 factors — portability and external adaptability):

- **Portability:** The effort required to transfer the program from one hardware/software platform to another.
- **Reusability:** The extent to which a program or its components can be reused in other applications.
- **Interoperability:** The effort required to couple one system with another.

Why this organization? McCall structured the model around 3 perspectives to reflect the complete operational lifecycle of military and industrial software: using it daily (Operation), fixing and evolving it (Revision), and migrating or integrating it into new computing environments (Transition).

ملاحظة أكاديمية هامة (Academic Note)

قاعدة الحفظ السريع لعوامل مكول (11 عاملاً): - التشغيل (5): Correctness, Reliability, Efficiency, Integrity, Usability. - التعديل (3): Maintainability, Flexibility, Testability. - الانتقال (3): Portability, Reusability, Interoperability.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.2, pp. 323–324; companion slides 30–35.

Question 7.9 · Factors vs Criteria in McCall's Model (Exercise 7.11)

Discuss the relationship between quality factors and quality criteria in McCall's software quality model.

In McCall's model, the relationship between quality factors and quality criteria represents a hierarchical decomposition from user-oriented quality desires down to developer-oriented software attributes:

- **Quality Factors (High-Level / External):** Represent external views of quality meaningful to end-users and management (e.g., *Is the software reliable?*, *Is it easy to maintain?*). Factors cannot be measured directly through a single inspection.
- **Quality Criteria (Intermediate / Internal):** Represent technical, internal software engineering attributes meaningful to software developers and architects (e.g., *Error tolerance, Simplicity, Modularity, Traceability, Consistency, Instrumentation*).

Structural Relationship: - **Multi-to-Multi Deconstructive Mapping: Each quality factor is evaluated through a composite set of related criteria.** For example, *Maintainability* is evaluated via *Simplicity, Conciseness, Self-descriptiveness, and Modularity*. - **Shared Technical Attributes:** Multiple factors often share underlying criteria. For instance, *Simplicity* contributes directly to *Maintainability, Testability, and Usability*. - **Measurement Bridge:** Software metrics measure technical criteria; the combined scores of criteria determine the assessment of the overarching quality factors.

ملاحظة أكاديمية هامة (Academic Note)

العلاقة: العامل (Factor) خارجي يراه المستخدم (مثل Maintainability) ← المعيار (Criterion) داخلي يراه المطور (مثل Simplicity و Modularity) ← القياسات (Metrics) تقيس المعايير لتحديد درجة تحقق العوامل.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.2, pp. 324–325; companion slides 35–39.

Question 7.10 · ISO 9126 Quality Model: Characteristics and Sub-Characteristics (Exercise 7.13)

What is ISO 9126? What are the quality characteristics and attributes?

ISO/IEC 9126 (established in 1991) is an international standard that defines an internationally harmonized software product quality evaluation model. It defines **6 top-level quality characteristics**, decomposed into **21 sub-characteristics (attributes)**:

1. **Functionality:**
2. *Suitability:* Appropriateness of functions for specified tasks.
3. *Accuracy:* Provision of right or agreed results or effects.
4. *Interoperability:* Ability to interact with specified systems.
5. *Compliance:* Adherence to application-related standards or conventions.
6. *Security:* Prevention of unauthorized access to programs and data.
7. **Reliability:**
8. *Maturity:* Frequency of failure by faults in the software.

9. *Fault Tolerance:* Ability to maintain a specified level of performance in cases of software faults.
10. *Recoverability:* Capability to re-establish performance level and recover data following a failure.
11. **Usability:**
12. *Understandability:* Effort required to recognize logical concepts and applicability.
13. *Learnability:* Effort required for learning application usage.
14. *Operability:* Effort required for operation and operation control.
15. **Efficiency:**
16. *Time Behaviour:* Response/processing times and throughput rates.
17. *Resource Behaviour:* Amount of resources used and duration of use.
18. **Maintainability:**
19. *Analyzability:* Effort needed for diagnosis of deficiencies or failure causes.
20. *Changeability:* Effort needed for modification, fault removal, or environmental change.
21. *Stability:* Risk of unexpected effects from modifications.
22. *Testability:* Effort needed for validating the modified software.
23. **Portability:**
24. *Adaptability:* Opportunity for software adaptation to different specified environments.
25. *Installability:* Effort needed to install software in a specified environment.
26. *Conformance:* Adherence to standards or conventions relating to portability.
27. *Replaceability:* Opportunity and effort to use this software in place of other software.

ملاحظة أكاديمية هامة (Academic Note)

مفتاح الحفظ لمعيار ISO 9126: يتألف من 6 خصائص عليا: Functionality, Reliability, Usability, Efficiency, Maintainability, Portability ← تتفرع إلى 21 سمة فرعية.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.2.4, pp. 326–328; companion slides 41–46.

Question 7.11 · Musa Basic Execution Time Model Formulation & Derivation (Exercise 7.15)

Discuss the basic model of software reliability. How can μ and τ be calculated?

The Basic Execution Time Model (developed by John Musa) assumes that the failure intensity decreases linearly with the number of failures experienced:

$$\lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0}\right)$$

where: - λ_0 is the initial failure intensity at the start of execution ($\mu = 0$). - v_0 is the total expected number of failures over infinite execution time. - μ is the cumulative number of failures experienced up to the current time.

Derivation of $\mu(\tau)$ and $\lambda(\tau)$ as Functions of Execution Time (τ): 1. Since failure intensity is the time rate of failure occurrences, $\lambda = \frac{d\mu}{d\tau}$:

$$\frac{d\mu}{d\tau} = \lambda_0 \left(1 - \frac{\mu}{v_0}\right) = \frac{\lambda_0}{v_0}(v_0 - \mu)$$

2. Separating variables and integrating from $\tau = 0$ (where $\mu = 0$) to τ:

$$\int_0^{\mu} \frac{d\mu}{v_0 - \mu} = \frac{\lambda_0}{v_0} \int_0^{\tau} d\tau$$

$$-\ln\left(\frac{v_0 - \mu}{v_0}\right) = \frac{\lambda_0}{v_0}\tau \implies \frac{v_0 - \mu}{v_0} = \exp\left(-\frac{\lambda_0}{v_0}\tau\right)$$

3. Solving for failures experienced $\mu(\tau)$:

$$\mu(\tau) = v_0 \left[1 - \exp\left(-\frac{\lambda_0}{v_0}\tau\right)\right]$$

4. Substituting $\mu(\tau)$ back into $\lambda(\mu)$ yields failure intensity over time:

$$\lambda(\tau) = \lambda_0 \exp\left(-\frac{\lambda_0}{v_0}\tau\right)$$

- **Additional failures:** $\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_P - \lambda_F)$.
- **Additional execution time:** $\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_P}{\lambda_F}\right)$.

ملاحظة أكاديمية هامة (Academic Note)

خلاصة النموذج الأساسي (Basic Model): العلاقة خطية مع عدد الأخطاء المكتشفة (μ) ← ولكنها أسية تناقصية مع زمن التنفيذ (τ).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.1, pp. 329–334; companion slides 47–58.

Question 7.12 · Logarithmic Poisson Execution Time Model & Increments (Exercise 7.19)

Write a short note on the Logarithmic Poisson Execution Time model. How can we calculate $\Delta\mu$ and $\Delta\tau$?

The Logarithmic Poisson Execution Time Model (developed by Musa and Okumoto) models software failure occurrences as a Non-Homogeneous Poisson Process (NHPP) where the failure intensity decreases exponentially with the number of failures experienced:

$$\lambda(\mu) = \lambda_0 \exp(-\theta\mu)$$

where: - λ_0 is the initial failure intensity. - θ is the **failure intensity decay parameter**, which measures the relative reduction in failure intensity per failure experienced.

Key Characteristics: - Unlike the Basic model, **the expected total number of failures over infinite time is infinite** ($v_0 = \infty$), reflecting the reality that software systems continually evolve and undergo modifications. - **Early failures cause a much larger decrease in failure intensity than later failures**,

which models non-uniform operational profiles where frequently executed code paths fail and are fixed first.

Formulas for $\mu(\tau)$ and $\lambda(\tau)$:

$$\mu(\tau) = \frac{1}{\theta}\ln(\lambda_0\theta\tau + 1)$$

$$\lambda(\tau) = \frac{\lambda_0}{\lambda_0\theta\tau + 1}$$

- **Additional failures:** $\Delta\mu = \frac{1}{\theta}\ln\left(\frac{\lambda_P}{\lambda_F}\right)$.
- **Additional execution time:** $\Delta\tau = \frac{1}{\theta}\left(\frac{1}{\lambda_F} - \frac{1}{\lambda_P}\right)$.

ملاحظة أكاديمية هامة (Academic Note)

مقارنة محورية بين النموذجين: - النموذج الأساسي (Basic): $\Delta\mu$ خطي، و $\Delta\tau$ لوغاريتمي. - النموذج اللوغاريتمي (Logarithmic): $\Delta\mu$ لوغاريتمي، و $\Delta\tau$ كسر معكوس مقلوب $\left(\frac{1}{\lambda_F} - \frac{1}{\lambda_P}\right)$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.2, pp. 335–339; companion slides 59–71.

Question 7.13 · Calendar Time Component Model & Time Conversion (Exercise 7.23)

Discuss the calendar time component model. Establish the relationship between calendar time to execution time.

The Calendar Time Component Model bridges the gap between software execution time (τ, measured in CPU hours) and managerially relevant calendar time (t, measured in elapsed working days or months).

Resource-Limited Segments: In any testing phase, progress is constrained by one of three limiting project resources: 1. **Computer Execution Time:** Limited by available test computers/workstations ($k = C$). 2. **Failure Identification Personnel:** Limited by available test engineers executing tests ($k = I$). 3. **Failure Correction Personnel:** Limited by debugging programmers diagnosing and fixing defects ($k = F$).

Mathematical Relationship between Calendar Time and Execution Time: Resource usage χ_k is modeled as a linear combination of execution time and failures experienced:

$$\chi_k = \theta_k\tau + \mu_k\mu$$

where θ_k is resource usage per unit execution time, and μ_k is resource usage per failure.

Differentiating with respect to execution time τ:

$$\frac{d\chi_k}{d\tau} = \theta_k + \mu_k\frac{d\mu}{d\tau} = \theta_k + \mu_k\lambda$$

The instantaneous conversion ratio from execution time to calendar time is obtained by dividing by available utilized resources $P_k\rho_k$:

$$\frac{dt}{d\tau} = \frac{1}{P_k \rho_k} \frac{d\chi_k}{d\tau} = \frac{\theta_k + \mu_k \lambda}{P_k \rho_k}$$

Integration over an interval:

$$\Delta t = \frac{\theta_k \Delta\tau + \mu_k \Delta\mu}{P_k \rho_k}$$

During testing, the governing bottleneck shifts dynamically from failure correction (high initial failure intensity) to failure identification, and finally to computer time as failures become rare.

ملاحظة أكاديمية هامة (Academic Note)

أهمية النموذج: زمن المعالج (CPU Hours) لا يفهمه مدير المشروع والزبون ← نموذج زمن التقويم يحول ساعات التنفيذ إلى أيام عمل وتاريخ تسليم حقيقي بناءً على المورد الأشد اختناقاً (Bottleneck).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.3, pp. 339–344; companion slides 72–88.

Question 7.14 · Jelinski–Moranda Reliability Model Formulation (Exercise 7.25)

Explain the Jelinski–Moranda model of reliability theory. What is the relation between t and λ?

The Jelinski–Moranda Model (1972) is one of the earliest and most fundamental software reliability growth models.

Underlying Assumptions: 1. The software contains a finite, fixed number of initial faults, denoted as N. 2. Each fault is independent and contributes equally to the overall failure hazard rate. 3. Whenever a failure occurs, the fault causing it is immediately and perfectly repaired with zero defect introduction. 4. The failure rate between successive failures remains constant.

Relation between Time (t) and Failure Rate (λ): - The hazard rate during the interval between the $(i-1)$-th and the i-th failure is a piecewise constant step function:

$$\lambda(t_i) = \phi[N - (i-1)]$$

where ϕ is a constant of proportionality representing the hazard rate per single unmanifested fault. - **Stepwise relation: The relationship between time t and hazard rate λ is a discrete step-down staircase curve.** As cumulative operational time t advances and failures occur, each repair decreases the hazard rate by a discrete step ϕ. The time interval between successive failures T_i grows stochastically larger as i approaches N.

ملاحظة أكاديمية هامة (Academic Note)

الخاصية الأساسية لنموذج Moranda–Jelinski: معدل الخطورة (Hazard rate) ثابت بين كل فشلين متتاليين، ويهبط هبوطاً درجياً (Staircase) بمقدار ϕ مع كل تصليح ناجح.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.4, pp. 343–346; companion slides 89–92.

Question 7.15 · Mills Bug Seeding Model Principles & Advantages (Exercise 7.26)

Describe the Mills bug seeding model. Discuss few advantages of this model over other reliability models.

The Mills Bug Seeding Model (introduced by Harlan Mills in 1972) is an empirical reliability estimation technique based on the classical capture-recapture ecological sampling method.

Operational Principle: 1. An independent quality officer intentionally inserts (seeds) a known number S of artificial bugs into the software codebase prior to testing. 2. The regular testing and debugging team conducts normal testing without knowledge of the count, types, or locations of the seeded bugs. 3. During testing, discovered bugs are classified into seeded bugs (s) and naturally occurring indigenous bugs (n). 4. Assuming seeded and indigenous bugs are equally distributed and equally detectable, the ratio of caught seeded bugs to total seeded bugs approximates the ratio of caught indigenous bugs to total indigenous bugs:

$$\frac{s}{S} \approx \frac{n}{N} \implies \hat{N} = \frac{S \times n}{s}$$

Advantages of Bug Seeding: - **Simple, intuitive mechanics:** Requires no complex statistical distribution fittings, differential equation solving, or continuous calendar time logging. - **Direct fault population estimation:** Estimates the absolute count of latent bugs remaining in the code ($\hat{N} - n$) directly. - **Tester efficiency metric:** The recovery ratio $\frac{s}{S}$ provides an immediate, objective measurement of test team thoroughness.

Primary Limitation: In practice, it is extremely difficult to synthesize artificial bugs that possess the exact error profile, subtlety, and execution probabilities of real human programming defects.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

معادلة ميلز الأساسية:

$$\hat{N} = \frac{S \times n}{s}$$

حيث S الأخطاء المزروعة، s المكتشف منها، n الأخطاء الأصلية المكتشفة، و $\hat{N}$ إجمالي الأخطاء الأصلية المقدرة.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.3.5, pp. 346–347; companion slide 93.

Question 7.16 · CMM Process Improvement Philosophy (Exercise 7.27)

Explain how the CMM encourages continuous improvement of the software process.

The Capability Maturity Model (CMM) is a five-level staged approach to software-process improvement, progressing from ad hoc work at Level 1 toward continuous process improvement at Level 5.

How CMM Encourages Continuous Improvement: 1. **Progressive levels:** Level 2 establishes basic project-management controls, Level 3 defines organization-wide processes, and Level 4 measures and controls process performance quantitatively. 2. **Continuous improvement:** At Level 5, the organization

proactively identifies weaknesses, analyzes defects to determine their causes, and uses the results to prevent known defect types from recurring. 3. **Common features:** Commitment to perform, ability to perform, activities performed, measurement and analysis, and verifying implementation describe conditions used to institutionalize key process areas.

ملاحظة أكاديمية هامة (Academic Note)

تدرّج CMM: المستوى الرابع للقياس والضبط الكمي؛ المستوى الخامس للتحسين المستمر والوقاية من تكرار أنواع العيوب المعروفة.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4 & §7.4.1, pp. 347–350; companion slides 94–96.

Question 7.17 · CMM Key Process Areas Across Maturity Levels (Exercise 7.28)

Discuss various key process areas of CMM at various maturity levels.

In the SEI Capability Maturity Model, **Key Process Areas (KPAs)** represent clusters of related activities that, when performed collectively, achieve a set of goals considered critical for enhancing process capability.

The 18 KPAs organized across the 5 Maturity Levels:

1. **Level 1: Initial**
2. **Zero KPAs.** (No formal KPAs; ad hoc, chaotic management).
3. **Level 2: Repeatable (6 KPAs — Basic Project Management Discipline):**
4. *Requirements Management (RM)*
5. *Software Project Planning (SPP)*
6. *Software Project Tracking and Oversight (SPTO)*
7. *Software Subcontract Management (SSM)*
8. *Software Quality Assurance (SQA)*
9. *Software Configuration Management (SCM)*
10. **Level 3: Defined (7 KPAs — Organization-Wide Standard Process):**
11. *Organization Process Focus (OPF)*
12. *Organization Process Definition (OPD)*
13. *Training Program (TP)*
14. *Integrated Software Management (ISM)*
15. *Software Product Engineering (SPE)*
16. *Intergroup Coordination (IC)*
17. *Peer Reviews (PR)*
18. **Level 4: Managed (2 KPAs — Quantitative & Statistical Process Control):**
19. *Quantitative Process Management (QPM)*
20. *Software Quality Management (SQM)*
21. **Level 5: Optimizing (3 KPAs — Continuous Process Improvement):**
22. *Defect Prevention (DP)*
23. *Technology Change Management (TCM)*

24. *Process Change Management (PCM)*

ملاحظة أكاديمية هامة (Academic Note)

توزيع مجالات العمل الرئيسية (KPAs): المستوى 1: (0) ← المستوى 2: (6) ← المستوى 3: (7) ← المستوى 4: (2) ← المستوى 5: (3) ← المجموع الكلي: 18 KPA.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4.2, pp. 350–353; companion slides 97–101.

Question 7.18 · Defined Level KPAs & Peer Review Activities (Exercise 7.33)

What are the various key process areas at defined level in CMM? Describe activities associated with one key process area.

The 7 Key Process Areas at Level 3 (Defined) are: 1. Organization Process Focus (OPF) 2. Organization Process Definition (OPD) 3. Training Program (TP) 4. Integrated Software Management (ISM) 5. Software Product Engineering (SPE) 6. Intergroup Coordination (IC) 7. Peer Reviews (PR)

Peer Reviews (PR) focus on removing defects from software work products early and efficiently. They also help participants understand the work products and the defects that can be prevented.

ملاحظة أكاديمية هامة (Academic Note)

فكرة Peer Reviews: مراجعة مبكرة لمخرجات العمل تساعد على كشف العيوب وفهمها قبل انتقالها إلى مراحل لاحقة.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4.2, p. 351; companion slides 98–100.
